

Centro Regional Universitario Córdoba IUA

Maestría en Ciberdefensa



Trabajo Práctico N°4

Secuencia cifrante repetida

Módulo:

- Criptografía

Alumno:

- Ing. Vietto Herrera Santiago

Docentes:

- Ing. Montes Miguel

18 de Mayo de 2026

Córdoba - Argentina

Indice

Desarrollo	1
1. Cifrados de flujo	1
1.1. Secuencia de uso único	1
1.2. Cifrados de flujo	2
1.2.1. Modelo general de un cifrado binario aditivo	2
1.2.2. Nonce	3
2. Solución del desafío	4
2.1. Análisis del problema	4
2.2. Implementación de la solución	5
2.2.1. Conexión con servidor, solicitud del desafío y extracción de los textos cifrados	5
2.2.2. Buscar la secuencia cifrante repetida	6
2.2.5. Envío de la respuesta al servidor	10

Desarrollo

1. Cifrados de flujo

1.1. Secuencia de uso único

Hemos mencionado que el único algoritmo de cifrado que ofrece seguridad incondicional es el One-Time Pad. Podemos generalizarlo de la siguiente manera:

- El alfabeto del mensaje son los enteros módulo m , es decir, el resto de dividir los enteros por un cierto valor. En la práctica, hacemos que m valga 0s o 1s.
- La clave o secuencia cifrante (keystream) es una secuencia de símbolos del mismo alfabeto, generada en forma aleatoria, independiente y uniforme.
- El emisor y receptor deben conocer la clave.
- Para cifrar, se usa la suma módulo m (la cual equivale a la operación $\oplus$ cuando $m = 2$, que es el XOR).

Supongamos por ejemplo que $m = 2$. Es decir, el alfabeto del mensaje es $Z_2 = \{0, 1\}$, y por lo tanto el mensaje es una secuencia de bits.

- Para cifrar una secuencia de bits $b = b_0, b_1, \dots, b_n$, hay que generar una clave aleatoria de la misma longitud $k = k_0, k_1, \dots, k_n$.
- El texto cifrado es $c = c_0, c_1, \dots, c_n$, donde $c_i = b_i \oplus k_i$

Por ejemplo, tenemos una secuencia de bits, donde generamos una secuencia realmente aleatoria de bits, y se va calculando XOR de cada elemento con la secuencia cifrante, y lo que obtenemos es el texto cifrado.

Para descifrar lo que hacemos es aplicar la misma secuencia cifrante al texto cifrado, y como la operación del XOR es su propia inversa, obtenemos nuevamente el texto claro.

	0	1	2	3	4	5	6	7
b	0	1	0	0	0	1	1	0
k	0	1	1	0	1	0	1	0
c	0	0	1	0	1	1	0	0

Las ventajas que ofrecen las secuencias de uso único es su seguridad incondicional. Pero como desventajas tenemos:

- Los elementos de la secuencia tienen que ser verdaderamente aleatorios, lo cual no es sencillo.
- La secuencia sólo se puede usar una vez. Si se vuelve a utilizar, el adversario puede, mediante una simple operación, eliminar la secuencia y proceder con técnicas normales de criptoanálisis.
- La longitud de la clave debe ser igual a la longitud del mensaje, lo que implica problemas tanto de distribución como de almacenamiento. Por lo tanto, no es un mecanismo práctico.
- Receptor y emisor deben estar perfectamente sincronizados.

Como consecuencia de esto surgen los cifrados de flujo, los cuales reemplazan la secuencia realmente aleatoria por una secuencia pseudoaleatoria originada en una clave. Podremos concebir a dicha clave como si fuera una semilla del generador de números pseudo aleatorios, que genera una secuencia de valores que podemos utilizar.

Esto implica renunciar a la seguridad incondicional, pero se espera que el algoritmo sea computacionalmente seguro.

1.2. Cifrados de flujo

Un cifrador de flujo genera una secuencia de símbolos pseudoaleatorios y los combina con los símbolos del texto claro mediante alguna operación binaria inversible sencilla.

Debemos entender que el texto claro, la secuencia cifrante, la clave y el texto cifrado son secuencias de bits, y la operación binaria es or-exclusivo. O podría ser por combinaciones de bits mas grandes como bytes, palabras de 32 bits, etc.

$$\begin{array}{rcccc}
 P & = & p_1 & p_2 & p_3 & \dots \\
 & & \oplus & \oplus & \oplus & \dots \\
 & & k_1 & k_2 & k_3 & \dots \\
 \hline
 C & = & c_1 & c_2 & c_3 & \dots
 \end{array}$$

Normalmente la operación que se utiliza para cifrar es el XOR:

$$c_i = p_i \oplus k_i$$

Para descifrar se realiza la misma operación porque la operación es su propia inversa. Es decir, se realiza el or-exclusivo entre el texto cifrado y la misma secuencia de bits.

$$p_i = c_i \oplus k_i$$

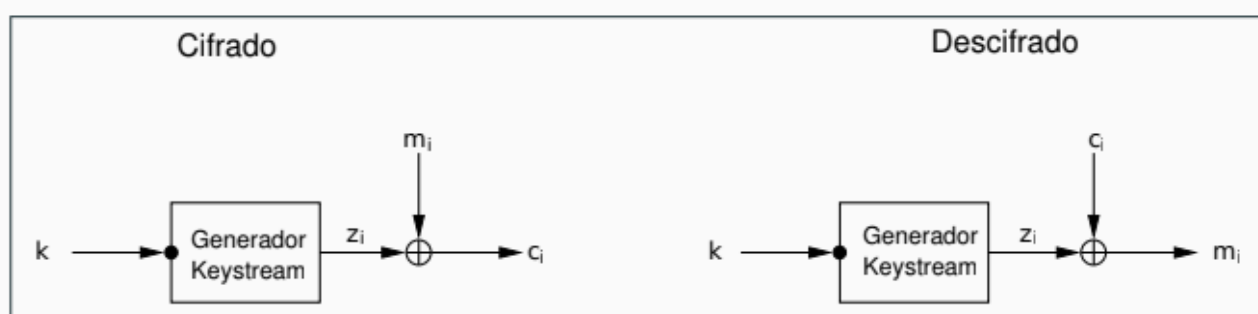
Al igual que con el one time pad. no debe repetirse la secuencia cifrante. Si se repite la secuencia cifrante toda la seguridad del algoritmo se rompe. Entonces un algoritmo actual, o bien regenera la clave una vez que hace falta o bien utiliza un nonce.

1.2.1. Modelo general de un cifrado binario aditivo

El modelo general mas sencillo seria el siguiente, en donde tenemos un algoritmo generador (Keystream) de secuencia cifrante, en donde el input es la clave, la salida es una secuencia cifrante la cual se le aplica un XOR con el mensaje, y con eso obtenemos cada uno de los elementos del texto cifrado.

Para descifrar tenemos la misma clave, el mismo generador que genera la misma secuencia cifrante, y desciframos.

El inconveniente de este esquema es que no podemos utilizar la misma clave dos veces. Porque la clave determina la secuencia cifrante de forma unívoca, y si utilizamos la misma clave dos veces, significa que vamos a producir dos veces la misma secuencia cifrante.

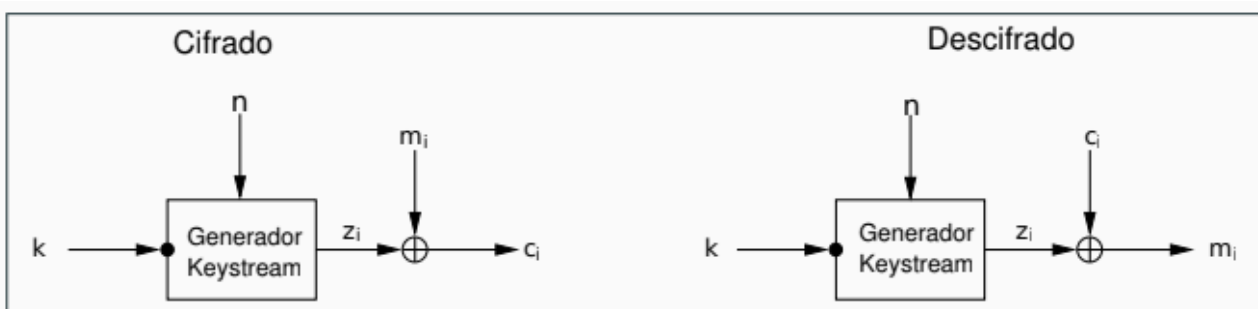


1.2.2. Nonce

Entonces, los algoritmos modernos requieren el uso de un nonce, lo cual permite reutilizar la clave siempre y cuando no se repita el nonce. La diferencia está en que el nonce no necesita ser secreto.

Nonce es una abreviatura de Number used only once. Este es un número que utilizamos asociado a un algoritmo y que no podemos repetir. En el contexto de los algoritmos de cifrado de flujo se suele llamar IV (por Initial Value).

Entonces, la variación de un algoritmo de cifrado de flujo con un nonce, es que tenemos el algoritmo generador de la secuencia cifrante que está parametrizado ahora por dos cosas, la clave y el nonce, que producen una secuencia cifrante que aplicamos al mensaje para cifrar. Y para descifrar con la clave y el nonce, se genera la misma secuencia cifrante y desciframos el mensaje.



El nonce solamente se necesita que sea distinto y no reutilizarlo, por lo tanto puede ser un contador, o un número de mensaje (mensaje 1, mensaje 2, etc), un número que debe ser distinto cada vez, porque en este contexto el nonce no requiere ser secreto, lo único secreto en todos estos sistemas es la clave.

Entonces, esto simplifica el problema porque no tenemos problemas en reutilizar la clave para cifrar muchos mensajes, siempre y cuando cada mensaje sea cifrado con un nonce distinto.

2. Solución del desafío

2.1. Análisis del problema

El siguiente trabajo práctico plantea el error más importante analizado anteriormente en la sección [1.2](#), es decir, reutilizar la misma secuencia cifrante en un cifrador de flujo.

En este caso, el servidor de la IUA entrega 5 textos cifrados, de los cuales hay 3 cifrados con la misma secuencia cifrante. Es decir, para esos tres mensajes tenemos algo como:

- $C1 = P1 \oplus K$
- $C2 = P2 \oplus K$
- $C3 = P3 \oplus K$

Donde K es la misma secuencia cifrante repetida. Por ende, el objetivo es encontrar K.

Esto es posible, porque si dos textos fueron cifrados con la misma secuencia cifrante:

- $C1 = P1 \oplus K$
- $C2 = P2 \oplus K$

Entonces, si hacemos XOR entre ambos cifrados:

- $C1 \oplus C2 = (P1 \oplus K) \oplus (P2 \oplus K)$

Como $K \oplus K = 0$, la secuencia cifrante desaparece:

- $C1 \oplus C2 = P1 \oplus P2$

Por lo tanto, esta es la falla que se busca explotar el práctico, es decir, la reutilización de la keystream permite eliminarla parcialmente del problema.

Además, los tres textos claros cifrados con la secuencia repetida tienen esta forma:

- Todos tienen la misma longitud.
- Todos usan caracteres ASCII imprimibles, sin espacios en blanco ni fines de línea, por ende sus códigos están entre el 33 al 126 inclusive.
- Dos textos están compuestos por repeticiones de un mismo carácter, por ejemplo AAAAAAAAAA o)))))))))).
- Uno de los textos es la representación decimal de un número par. Por ejemplo 12345678.

Como estrategia de solución, tenemos la información entonces de que como dos de los textos claros son repeticiones de un único carácter, podemos probar todas las posibilidades. Por ejemplo, Si un texto claro fuera AAAAAAAAAA, y tenemos su cifrado C, entonces la keystream candidata sería:

- $K = C \oplus AAAAAAAAAA$

Como no sabemos cuál carácter era, probamos todos los caracteres imprimibles ASCII del 33 hasta el 126.

Entonces, para cada texto cifrado, probamos:

- $K \text{ candidata} = \text{cifrado} \oplus \text{caracter_repetido}$

Después, usamos esa K candidata para descifrar los otros textos. Si la keystream es correcta, debería aparecer:

- Otro texto de un solo carácter repetido.
- Un número decimal par.

2.2. Implementación de la solución

2.2.1. Conexión con servidor, solicitud del desafío y extracción de los textos cifrados

Dado a lo entendido en la sección 2.1, como primer paso tenemos la solicitud del desafío al servidor:

1. Se definen las credenciales de acceso al servidor, que son por un lado el correo electrónico registrado del alumno que lleva a cabo el siguiente trabajo práctico, y la URL del servidor.

```
7
8 server = "https://cripto.iua.edu.ar"
9 email = "santiagovietto5@gmail.com"
10
```

2. Luego, se declara una función `get_challenge()` para hacer posible la comunicación con el servidor y solicitar a este el desafío mediante un GET, .

```
19
20 def get_challenge():
21     response = requests.get(f"{server}/timerand/{email}/challenge")
22     response.raise_for_status()
23     return response.text
24
```

3. Hacemos un llamado a la función `get_challenge()` almacenando el contenido en una variable `challenge`. Y mostramos el contenido obtenido por pantalla.

```
57
58 challenge = get_challenge()
59
60 ciphertxts = separate_challenge(challenge)
61
62 print("\nRespuesta del servidor:")
63 print(challenge)
64
65 print("Textos cifrados recibidos:", len(ciphertxts))
66 print("Longitud:", len(ciphertxts[0]))
67
```

```

• (venv_TPsCrypto) santiago-vietto@santiago-vietto-pc:~/Documents/Trabajos-Practicos-Criptografia/Unidad_3/TP4$

Respuesta del servidor:
xDP49hBQdEfN57pnUrBay6MMRYbc179VxoyJelpiOTpm8aiCU6YB
5ferif7STiLA1YKVl9/Bt/bLdauvV1Dlpwf8wzWTVBg5Aga36B3K
SEVA7V355co5qh7e6qefL7IF41Uh0yA3cY/RycHrgx/TtdNGRA0q
JCpsgTGViaZVxnKyhsvz+95pjzLNV0xbHe09pa2H730/2b8qKGFG
Q0sK7Lbx5sQyqRHU6auWmbAL4V0pNys9fYDcxcjpx3dut9NSQ8o

```

4. Por otro lado, se guarda en una variable `ciphertext` los textos cifrados separados y decodificados de Base64, mediante una función `separate_challenge()` declarada previamente. Esta toma la respuesta del servidor, la separa línea por línea, decodifica cada una desde Base64 y guarda cada texto cifrado como bytes. Esto es necesario porque para aplicar XOR debemos trabajar con bytes reales y no con Base64.

```

20
21 def separate_challenge(challenge):
22     lines = challenge.strip().splitlines()
23     ciphertexts = [base64.b64decode(line) for line in lines]
24     return ciphertexts
25

```

5. Se imprime por pantalla la cantidad de textos cifrados obtenidos y la longitud del primer texto cifrado ya decodificado. Como la consigna indica que los textos tienen la misma longitud, esta sirve como referencia para trabajar con todos.

```

Textos cifrados recibidos: 5
Longitud: 39

```

2.2.2. Buscar la secuencia cifrante repetida

En este paso el objetivo es encontrar cuál fue la keystream reutilizada para cifrar tres de los cinco textos.

1. Se define una variable `keystream_found` en `None` para determinar si la búsqueda de la secuencia cifrante correcta tuvo éxito o no. Cuando se encuentre una candidata que cumpla con las condiciones de la consigna, va guardar la secuencia cifrante encontrada.

2. En el ciclo `for` general, se recorren los 5 textos cifrados recibidos del servidor. Como no se sabe cuál de ellos corresponde a uno de los textos claros repetido, el código prueba con todos, tomando uno como base en cada recorrido. Entonces, en cada iteración, toma un texto cifrado como `base_ciphertext` y supone que ese cifrado podría provenir de un texto claro formado por un mismo carácter repetido.

2.1. Como los textos claros están formados por caracteres ASCII imprimibles entre 33 y 126, el ciclo `for` prueba cada posible carácter dentro del rango. Entonces, para cada texto cifrado tomado como base, el programa prueba todos los caracteres ASCII imprimibles posibles. Esto se hace porque la actividad indica que dos de los textos claros están formados por repeticiones de un único carácter, pero no sabemos cuál es ese carácter. Por lo tanto, se prueban hipótesis como `AAAAA...`, `:::...`, `VVVVV...`, etc.

2.2. Luego, con cada carácter probado, se construye un posible texto claro repetido. Este texto tiene la misma longitud que el cifrado base, es decir, 39, porque en un cifrado de flujo el texto claro, el texto cifrado y la keystream deben tener la misma longitud para poder aplicar XOR byte a byte. Entonces, cuando el carácter probado fue :, la ejecución construyó algo equivalente a:

-:

2.3. A partir del texto cifrado base y el texto claro supuesto construido, se calcula la keystream candidata aplicando XOR entre ambos. Esto se hace porque sabemos que el cifrado de flujo es $C = P \oplus K$, entonces, si tenemos el cifrado C y suponemos un posible texto claro P, podemos despejar la keystream $K = C \oplus P$.

Usando el resultado real, el razonamiento fue:

- base_ciphertext = C2
- repeated_plaintext =:
- candidate_keystream = C2 $\oplus$:

Pero en este punto todavía no sabemos si esa keystream es correcta. Es solamente una candidata. Para realizar el cálculo XOR utilizamos una función xor_bytes() declarada previamente.

```
26
27 def xor_bytes(a, b):
28     return bytes(x ^ y for x, y in zip(a, b))
29
```

2.4. Después de calcular una keystream candidata, se intenta descifrar los cinco textos cifrados con ella. Porque la idea es comprobar si esa keystream también sirve para otros textos. Básicamente para cada cifrado se aplica XOR con la keystream candidata, intentando obtener un posible texto claro:

- P0 = C0 $\oplus$ candidate_keystream
- P1 = C1 $\oplus$ candidate_keystream
- P2 = C2 $\oplus$ candidate_keystream
- P3 = C3 $\oplus$ candidate_keystream
- P4 = C4 $\oplus$ candidate_keystream

Si la keystream candidata es la correcta, al aplicarla sobre los textos cifrados que fueron cifrados con la misma secuencia deberían aparecer textos claros coherentes. Si la keystream candidata es incorrecta, lo más probable es que los resultados sean basura, caracteres raros o bytes no imprimibles. Por eso se valida con la función is_printable_ascii(), la cual solo guarda los resultados que tienen caracteres imprimibles, y además revisa que todos los bytes estén entre 33 y 126.

```
30
31 def is_printable_ascii(data):
32     return all(33 ≤ byte ≤ 126 for byte in data)
33
```

En la ejecución, con la keystream correcta aparecieron estos candidatos:

- [illegible]

Es decir, la keystream candidata no solo sirvió para el texto base 2, sino también para los textos 3 y 4.

2.5. Después de obtener los textos descifrados candidatos, el programa los clasifica en dos grupos. El primer grupo guarda textos formados por un único carácter repetido, y el segundo grupo guarda textos que representan un número decimal par.

La función `is_repeated_character()` declarada previamente, convierte el texto en un conjunto con. Si todos los caracteres son iguales, el conjunto tendrá un solo elemento. Por ejemplo, “.....”, produce un conjunto con un solo valor {}, entonces se considera repetido. En cambio, si tenemos “ABCABC”, esto produce varios valores distintos, por lo que no se considera repetición de un único carácter.

La función `is_even_decimal_number()`, declarada previamente, verifica dos cosas, por un lado que todos los caracteres sean dígitos del 0 al 9, y por otro que el último dígito sea par. Por ejemplo 170912941950963484822610657638684561788 es válido porque está compuesto solo por dígitos y termina en 8, que es par.

En la ejecución, se clasificó correctamente:

- 2: (texto repetido)
- 3: VVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVV (texto repetido)
- 4: 170912941950963484822610657638684561788 (número decimal par)

```

34
35 def is_repeated_character(data):
36     return len(set(data)) == 1 and is_printable_ascii(data)
37
38
39 def is_even_decimal_number(data):
40     if not data:
41         return False
42
43     if not all(48 ≤ byte ≤ 57 for byte in data):
44         return False
45
46     return data[-1] in [
47         ord("0"),
48         ord("2"),
49         ord("4"),
50         ord("6"),
51         ord("8")
52     ]

```

2.6. La keystream candidata se considera válida solamente si al descifrar los textos aparecen los tres patrones esperados por la consigna, es decir, dos textos de caracteres repetidos y un número decimal par. Además, se verifica que estos patrones corresponden a tres textos cifrados distintos, ya que el desafío indica que son tres mensajes diferentes los que fueron cifrados con la misma secuencia cifrante. Entonces no alcanza con que un mismo texto cumpla más de una condición. Se busca que haya tres posiciones distintas involucradas.

En este caso, tenemos:

- `repeated_indexes = {2, 3}`
- `decimal_indexes = {4}`

Al unirlos:

- $\{2, 3\} \cup \{4\} = \{2, 3, 4\}$

La cantidad de índices distintos es 3, y por eso la keystream se acepta como correcta.

2.7. Cuando una keystream candidata cumple todas las condiciones, se almacena en `keystream_found` y se muestran los datos que permitieron encontrarla, es decir, el índice del texto cifrado usado como base, el carácter repetido supuesto y los textos descifrados candidatos. En la ejecución realizada, la solución se encontró tomando como base el texto cifrado 2 y suponiendo el carácter “.”. Al aplicar esa keystream, se obtuvieron dos textos repetidos y un número decimal par, por lo que se confirmó que era la secuencia cifrante reutilizada.

3. Una vez encontrada la keystream correcta, ya no tiene sentido seguir probando, por lo que se termina el ciclo. Y a la keystream encontrada, se la transforma a Base64 para poder enviar el resultado al servidor.

```
71
72 keystream_found = None
73
74 for base_index, base_ciphertext in enumerate(ciphertexts):
75
76     for character in range(33, 127):
77
78         repeated_plaintext = bytes([character]) * len(base_ciphertext)
79
80         candidate_keystream = xor_bytes(base_ciphertext, repeated_plaintext)
81
82         decrypted_candidates = []
83
84         for i, ciphertext in enumerate(ciphertexts):
85             plaintext = xor_bytes(ciphertext, candidate_keystream)
86
87             if is_printable_ascii(plaintext):
88                 decrypted_candidates.append((i, plaintext))
89
90         repeated_plaintexts = []
91         even_decimal_plaintexts = []
92
93         for i, plaintext in decrypted_candidates:
94             if is_repeated_character(plaintext):
95                 repeated_plaintexts.append((i, plaintext))
96
97             if is_even_decimal_number(plaintext):
98                 even_decimal_plaintexts.append((i, plaintext))
99
```


3) Finalmente, se imprime la respuesta del servidor, lo que permite ver si el envío fue correcto, mostrando el código de estado de la petición http al servidor, por ejemplo, código 200 como en este caso que salió bien, y el contenido de `response_answer.text` que muestra el mensaje que devuelve el servidor, en este caso ¡Ganaste!, indicando que la actividad fue resuelta correctamente.

```
Respuesta del servidor:  
Status code: 200  
¡Ganaste!
```